

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA10 / CSA1063	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:		Reg. No.:	
Date:		Duration:	60 minutes

Code of Conduct

I Ajay Velmail E (192311385) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ E Ajay Velmail _____

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

3. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

4. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

5. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

6. **Commit History Analysis**
 - Present the commit history of the project.
 - Explain how commit messages follow good version control practices and contribute to project maintenance.
7. **Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
8. **Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
9. **Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
10. **Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
11. **Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
12. **Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

CO3 ASSESSMENT TOOL 1 – GIT & DOCKER TECHNICAL ASSIGNMENT

Project: Cold-Chain Logistics Integrity Platform

1. PROBLEM ANALYSIS

1.1 Problem Statement

The **Cold-Chain Logistics Integrity Platform** is a distributed system designed to continuously monitor refrigerated shipments used for food and pharmaceutical distribution.

Products such as vaccines, insulin, biologics, dairy products, seafood, meat, and fresh produce require controlled environmental conditions throughout transportation and storage. Temperature excursions can occur because of refrigeration failures, power outages, door openings, traffic delays, or improper handling.

Traditional monitoring systems are often reactive because data may only be reviewed after delivery. This makes it difficult to intervene while a shipment is still in transit.

The proposed platform addresses this problem by continuously collecting:

- Temperature
- Humidity
- GPS location
- Door status
- Refrigeration-unit health
- Shipment information

The system detects environmental excursions, predicts potential failures, sends alerts, and generates compliance reports.

1.2 Project Objectives

The main objectives are:

1. Continuously collect IoT sensor data.
2. Monitor shipment conditions in real time.
3. Detect environmental deviations.
4. Apply product-specific thresholds.
5. Generate alerts immediately.
6. Predict potential refrigeration failures.
7. Track shipment location.
8. Maintain historical records.
9. Generate compliance documentation.
10. Maintain audit trails.

11. Support thousands of connected devices.
12. Provide secure and reliable operation.

1.3 Functional Requirements

ID	Functional Requirement
FR01	User authentication and authorization
FR02	IoT sensor registration
FR03	Real-time telemetry collection
FR04	Temperature monitoring
FR05	Humidity monitoring
FR06	GPS tracking
FR07	Door-status monitoring
FR08	Refrigeration-unit monitoring
FR09	Product-specific threshold configuration
FR10	Environmental excursion detection
FR11	Real-time alert generation
FR12	Shipment management
FR13	Historical data analysis
FR14	Predictive failure detection
FR15	Compliance report generation
FR16	Audit trail maintenance

1.4 Non-Functional Requirements

Scalability

The system should support thousands of simultaneously connected IoT devices.

Performance

Telemetry should be processed with low latency so that critical excursions can be identified quickly.

Availability

The platform should remain operational continuously because cold-chain monitoring cannot depend on normal business hours.

Reliability

Sensor data should be received, processed, and stored reliably.

Security

Communication between IoT devices, backend services, and users should be protected.

Maintainability

The application should have a modular structure so that individual components can be modified independently.

Portability

The application should run consistently across development, testing, and production environments.

1.5 Expected Outcomes

The platform is expected to:

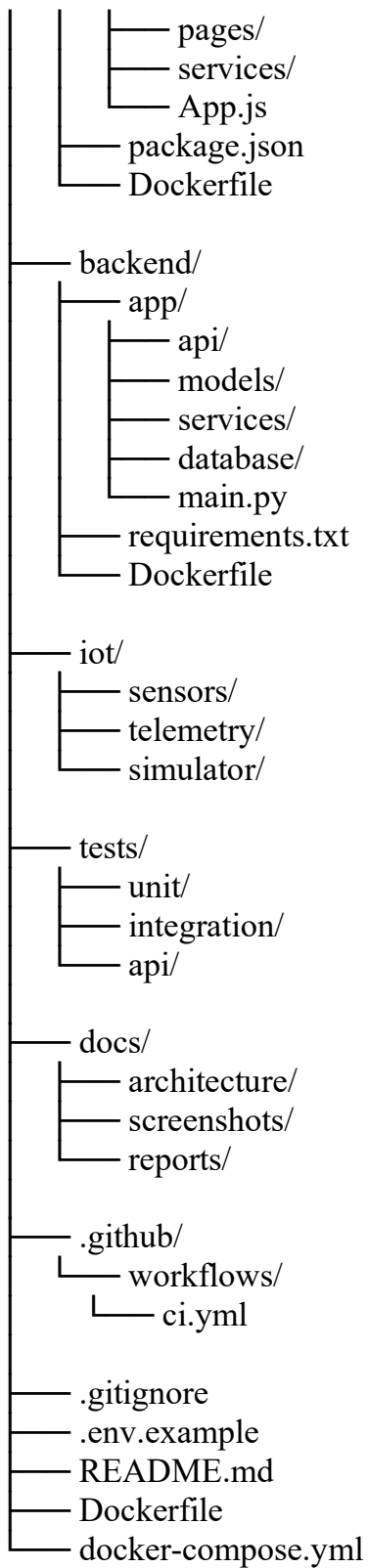
- Reduce product spoilage.
- Reduce financial losses.
- Improve regulatory compliance.
- Enable proactive intervention.
- Improve supply-chain visibility.
- Reduce manual monitoring.
- Improve customer trust.
- Support large logistics fleets.

2. PROJECT STRUCTURE DESIGN

A well-organized repository is important for maintaining the application source code, Git configuration, Docker configuration, documentation, tests, and deployment files.

2.1 Proposed Folder Structure

```
cold-chain-platform/  
├── frontend/  
│   ├── public/  
│   ├── src/  
│   └── components/
```



2.2 Folder Justification

frontend/

Contains the user interface.

It provides:

- Shipment dashboard
- Temperature visualization
- Alerts
- GPS information
- User interface

backend/

Contains the main application logic.

It handles:

- APIs
- Authentication
- Shipment management
- Telemetry processing
- Threshold checking
- Alerts
- Database operations

iot/

Contains IoT-related functionality.

It can contain:

- Sensor communication
- Telemetry generation
- Sensor simulation
- Device-related code

tests/

Contains automated tests.

unit/ → Individual component testing
integration/ → Service integration testing
api/ → API endpoint testing

docs/

Contains project documentation, architecture diagrams, screenshots, and reports.

.github/

Contains GitHub Actions workflows for automated CI/CD.

.gitignore

Specifies files that should not be tracked by Git.

Examples:

```
.env  
__pycache__/  
node_modules/  
*.log  
.venv/
```

.env.example

Provides example environment variables without exposing actual secrets.

Example:

```
DATABASE_URL=  
API_KEY=  
SECRET_KEY=
```

README.md

Provides instructions for:

- Project setup
- Installation
- Git workflow
- Docker execution
- API usage
- Testing

Dockerfile

Defines how an application component is packaged into a Docker image.

docker-compose.yml

Defines and manages multiple containers required by the application.

3. GIT REPOSITORY INITIALIZATION

Git is used as the version-control system for tracking source-code changes and enabling collaborative development.

3.1 Initialize the Repository

Create the project directory:

```
mkdir cold-chain-platform  
cd cold-chain-platform
```

Initialize Git:

```
git init
```

Check repository status:

```
git status
```

3.2 Configure Git Identity

```
git config user.name "Your Name"  
git config user.email "your@email.com"
```

3.3 Create Initial Files

```
touch README.md  
touch .gitignore  
touch docker-compose.yml
```

Then add the project files:

```
git add .
```

Create the initial commit:

```
git commit -m "chore: initialize cold-chain platform"
```

3.4 Connect to Remote Repository

Create a remote repository and connect it:

```
git remote add origin <repository-url>
```

Verify:

```
git remote -v
```

Push the project:

```
git push -u origin main
```

3.5 Essential Git Files

File

Purpose

.git/	Stores Git repository metadata and history
.gitignore	Prevents unnecessary/sensitive files from being tracked
README.md	Contains project documentation
.gitattributes	Defines Git-specific file attributes
.gitconfig	Stores Git configuration at user/system level

Important

The .git/ directory is automatically created by:

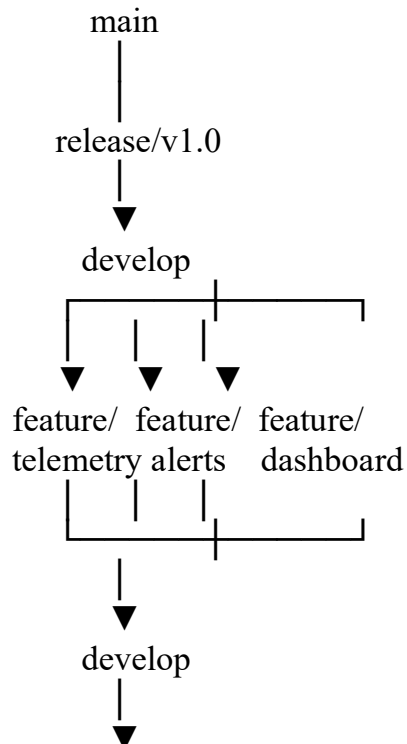
`git init`

It contains the internal information required by Git to manage versions, branches, commits, and repository history.

4. GIT BRANCHING STRATEGY

4.1 Selected Strategy

For the Cold-Chain Logistics Platform, a **feature-based Git branching strategy with main and develop branches** is suitable.



main

4.2 Main Branch

The main branch contains stable, production-ready code.

`git checkout main`

Only tested and approved code should be merged into main.

4.3 Develop Branch

The develop branch contains integrated development work.

`git checkout -b develop`

Features are merged here before release.

4.4 Feature Branches

Each feature is developed independently.

Example:

`git checkout -b feature/telemetry-monitoring`

Other examples:

`feature/sensor-registration`

`feature/alert-system`

`feature/gps-tracking`

`feature/compliance-report`

`feature/dashboard`

4.5 Release Branch

Used to prepare a stable version.

```
git checkout -b release/v1.0
```

It can be used for:

- Final testing
- Bug fixes
- Documentation
- Version preparation

4.6 Hotfix Branch

Used for urgent production issues.

```
git checkout -b hotfix/temperature-alert
```

After fixing the problem, the hotfix is merged into the stable branch and development branch where appropriate.

4.7 Why This Strategy is Suitable

This strategy provides:

- Parallel development
- Controlled releases
- Reduced risk to production code
- Easier collaboration
- Clear feature ownership
- Better project organization
- Easier bug fixing

For a distributed cold-chain platform containing several independent components, feature branches allow developers to work without interfering with each other's changes.

5. VERSION CONTROL WORKFLOW

5.1 Create a Feature Branch

```
git checkout develop  
git pull origin develop  
git checkout -b feature/telemetry-monitoring
```

5.2 Check Changes

```
git status
```

5.3 Stage Changes

```
git add backend/
```

5.4 Commit Changes

```
git commit -m "feat: add real-time telemetry ingestion"
```

5.5 Push Feature Branch

```
git push -u origin feature/telemetry-monitoring
```

5.6 Merge Feature

After testing:

```
git checkout develop  
git pull origin develop  
git merge feature/telemetry-monitoring
```

5.7 Delete Feature Branch

```
git branch -d feature/telemetry-monitoring
```

5.8 Synchronize Repository

```
git pull origin develop
git push origin develop
```

5.9 Conflict Resolution

A conflict may occur when two developers modify the same section of a file.

Git may display:

```
<<<<<<< HEAD
Current branch code
=====
Incoming branch code
>>>>>>> feature/alert-system
```

The developer manually selects the correct implementation and removes the conflict markers.

Then:

```
git add .
git commit -m "fix: resolve merge conflict in alert service"
```

5.10 Important Git Commands

Command	Purpose
git init	Initializes repository
git status	Displays working-tree status
git add	Stages changes
git commit	Saves changes to history
git branch	Manages branches
git checkout	Switches branches
git switch	Modern branch-switching command
git merge	Combines branch changes

git pull	Downloads and integrates remote changes
git push	Uploads local commits
git log	Displays commit history
git diff	Displays code differences
git clone	Copies a remote repository

6. COMMIT HISTORY ANALYSIS

A good commit history makes the project easier to understand, maintain, debug, and review.

6.1 Example Commit History

a81c3d2 feat: add telemetry ingestion service
b729d10 feat: implement temperature threshold validation
c451ef8 feat: add shipment monitoring dashboard
d921aa3 feat: implement critical alert notifications
e184bf2 fix: handle invalid sensor telemetry
f781c20 docs: update Docker deployment instructions
g551ab7 test: add telemetry integration tests
h120cd9 chore: initialize project structure

6.2 Good Commit Message Format

A professional convention can be:

<type>: <short description>

Examples:

feat: add GPS tracking service
fix: resolve temperature threshold validation
docs: update deployment instructions
test: add sensor telemetry tests
refactor: simplify alert processing
chore: update Docker dependencies

6.3 Common Commit Types

Type	Purpose
feat	New functionality
fix	Bug correction
docs	Documentation
test	Testing
refactor	Code restructuring
chore	Maintenance
perf	Performance improvement

6.4 Benefits of Good Commit History

A good commit history:

1. Makes changes easy to understand.
2. Helps identify when bugs were introduced.
3. Makes rollback easier.
4. Improves team collaboration.
5. Provides project traceability.
6. Helps reviewers understand development progress.
7. Supports long-term maintenance.

7. DOCKER ENVIRONMENT DESIGN

7.1 Why Docker is Suitable

Docker is suitable for this project because the platform contains multiple components that require different environments.

For example:

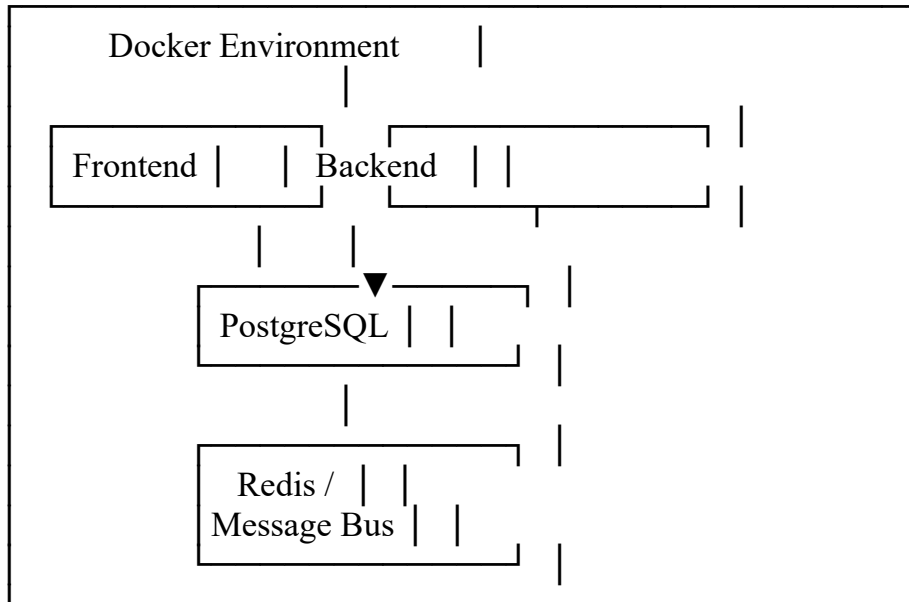
Frontend
Backend API
Database
Message Broker
IoT Simulator

Without containerization, configuring these components manually on every development machine can result in dependency and environment problems.

Docker packages applications together with their required runtime environment.

7.2 Components Requiring Containerization

The proposed system can use the following containers:



7.3 Docker Benefits

Docker provides:

- Environment consistency
- Portability
- Easy deployment
- Dependency isolation
- Faster development setup
- Service isolation
- Easier testing
- Simplified scaling

8. DOCKERFILE DEVELOPMENT

A Dockerfile defines how a Docker image is built.

For the backend, a Python-based application can use the following example.

8.1 Backend Dockerfile

```
FROM python:3.12-slim
```

```
WORKDIR /app
```

```
COPY requirements.txt .
```

```
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY app ./app
```

```
EXPOSE 8000
```

```
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

8.2 Dockerfile Explanation

FROM

```
FROM python:3.12-slim
```

Provides the base Python environment.

WORKDIR

```
WORKDIR /app
```

Sets /app as the working directory.

COPY

```
COPY requirements.txt .
```

Copies the dependency file into the container.

RUN

`RUN pip install --no-cache-dir -r requirements.txt`

Installs Python dependencies.

Second COPY

`COPY app ./app`

Copies application source code into the image.

EXPOSE

`EXPOSE 8000`

Documents the port used by the backend application.

CMD

`CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]`

Starts the backend application when the container runs.

8.3 Build Docker Image

`docker build -t cold-chain-backend .`

8.4 Run Docker Container

```
docker run -p 8000:8000 cold-chain-backend
```

9. DOCKER COMPOSE DESIGN

Docker Compose is useful because the project consists of multiple interconnected services.

9.1 Example docker-compose.yml

services:

backend:

build: ./backend

container_name: cold-chain-backend

ports:

- "8000:8000"

environment:

DATABASE_URL: postgresql://coldchain:password@db:5432/coldchain

depends_on:

- db

- redis

networks:

- coldchain-network

frontend:

build: ./frontend

container_name: cold-chain-frontend

ports:

- "3000:3000"

depends_on:

- backend

networks:

- coldchain-network

db:

image: postgres:16

container_name: cold-chain-db

environment:

POSTGRES_DB: coldchain

POSTGRES_USER: coldchain

POSTGRES_PASSWORD: password

volumes:

- postgres-data:/var/lib/postgresql/data

```
networks:  
  - coldchain-network
```

```
redis:  
  image: redis:7  
  container_name: cold-chain-redis  
  networks:  
    - coldchain-network
```

```
volumes:  
  postgres-data:
```

```
networks:  
  coldchain-network:  
    driver: bridge
```

9.2 Service Explanation

Backend

Runs the main application/API.

Port: 8000

Frontend

Provides the web-based user interface.

Port: 3000

PostgreSQL

Stores persistent application data.

Examples:

- Users
- Shipments

- Sensors
- Alerts
- Thresholds

Redis

Can be used for:

- Caching
- Temporary data
- Session information
- High-speed operations

It can also be replaced or supplemented by a dedicated event/message broker in a production deployment.

9.3 Networking

All services are connected to:

`coldchain-network`

This allows containers to communicate using service names.

For example, the backend can connect to PostgreSQL using:

`db:5432`

rather than `localhost`.

9.4 Volumes

volumes:

- `postgres-data:/var/lib/postgresql/data`

The volume ensures that database data persists even if the PostgreSQL container is removed or recreated.

9.5 Environment Variables

Environment variables prevent configuration values from being hardcoded directly into application code.

Example:

```
DATABASE_URL
POSTGRES_DB
POSTGRES_USER
POSTGRES_PASSWORD
```

For a real deployment, sensitive credentials should be managed securely rather than committed to Git.

9.6 Service Dependencies

depends_on:

- db
- redis

This expresses that the backend depends on the database and Redis services.

10. CONTAINER DEPLOYMENT AND TESTING

10.1 Build and Start Services

Run:

```
docker compose build
```

Then:

```
docker compose up -d
```

10.2 Check Running Containers

```
docker compose ps
```

Expected result:

NAME	STATUS
cold-chain-backend	running
cold-chain-frontend	running
cold-chain-db	running
cold-chain-redis	running

10.3 View Logs

```
docker compose logs
```

For a particular service:

```
docker compose logs backend
```

10.4 Test Backend

Open:

<http://localhost:8000>

If using FastAPI, the API documentation can be available at:

<http://localhost:8000/docs>

10.5 Test Frontend

Open:

<http://localhost:3000>

The cold-chain monitoring dashboard should be displayed.

10.6 Functional Testing

Test Case 1 – User Login

Input: Valid username and password

Expected Result: User successfully enters dashboard.

Test Case 2 – Sensor Registration

Input: Valid sensor ID and configuration.

Expected Result: Sensor is registered successfully.

Test Case 3 – Normal Temperature

Input:

Temperature = within configured range

Expected Result:

Status = NORMAL

Alert = NO

Test Case 4 – Temperature Excursion

Input:

Temperature = outside configured range

Expected Result:

Status = EXCURSION

Alert = GENERATED

Test Case 5 – Database Persistence

Create shipment data and restart the database container.

Expected Result: Data remains available because PostgreSQL uses a persistent Docker volume.

10.7 Useful Docker Commands

Command	Purpose
docker images	Display Docker images
docker ps	Display running containers
docker ps -a	Display all containers
docker build	Build an image
docker run	Run a container
docker stop	Stop a container
docker start	Start a container
docker logs	View container logs
docker exec	Execute command inside container
docker compose up	Start Compose services
docker compose down	Stop and remove services
docker compose ps	Display Compose service status

10.8 Screenshots to Include in the Report

For maximum marks, include screenshots of actual execution.

Screenshot 1

Git repository initialization:

git init

git status

Screenshot 2

Branch creation:

```
git branch
```

Screenshot 3

Feature branch:

```
git checkout -b feature/telemetry-monitoring
```

Screenshot 4

Commit history:

```
git log --oneline --graph --all
```

Screenshot 5

Successful Git push:

```
git push origin develop
```

Screenshot 6

Docker image:

```
docker images
```

Screenshot 7

Running containers:

```
docker compose ps
```

Screenshot 8

Docker logs:

docker compose logs backend

Screenshot 9

Frontend dashboard running.

Screenshot 10

Backend API / Swagger documentation running.

11. BENEFITS OF GIT AND DOCKER

11.1 Git Benefits

Collaboration

Multiple developers can work on separate branches simultaneously.

Version Management

Every modification is recorded in the Git history.

Branching

Features can be developed independently.

Rollback

Previous versions can be restored when necessary.

Traceability

Each commit identifies what was changed and why.

Code Review

Branches and pull requests allow changes to be reviewed before integration.

11.2 Docker Benefits

Portability

The same container can run across different environments.

Environment Consistency

Developers use consistent application dependencies.

Isolation

Each service runs in its own isolated environment.

Easy Deployment

The application can be started using:

```
docker compose up -d
```

Scalability

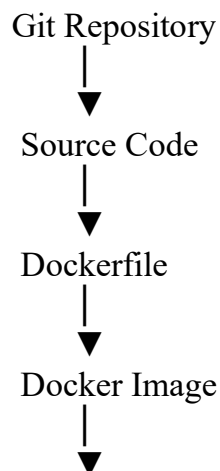
Services can be replicated and deployed independently.

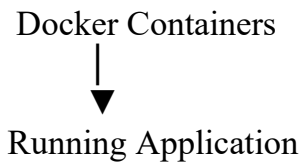
Faster Development

New developers can set up the project without manually installing every dependency.

11.3 Git + Docker Together

Git and Docker complement each other.





Git manages the **version of the application**, while Docker manages the **environment in which the application runs**.

Together they improve:

- Collaboration
- Reproducibility
- Deployment
- Portability
- Maintainability
- Version control
- Testing

12. CHALLENGES AND IMPROVEMENTS

12.1 Challenges

1. Dependency Management

Different services may require different versions of libraries and runtimes.

Solution: Use Dockerfiles to isolate dependencies.

2. Git Merge Conflicts

Multiple developers modifying the same files can result in conflicts.

Solution: Use feature branches, frequent synchronization, and clear ownership.

3. Large IoT Data Volume

Thousands of sensors can generate a very large volume of telemetry.

Solution: Use asynchronous event processing, scalable storage, and independent service scaling.

4. Container Networking

Services must communicate correctly across containers.

Solution: Use Docker Compose networks and service names.

5. Data Persistence

Container deletion can result in data loss if persistent storage is not configured.

Solution: Use Docker volumes for database storage.

6. Secret Management

Passwords and API keys should not be committed to Git.

Solution: Use environment variables and secure secret-management systems.

7. Debugging Distributed Services

Finding problems across several containers can be difficult.

Solution: Implement centralized logging, health checks, and monitoring.

8. Configuration Management

Development and production environments may have different configurations.

Solution: Use environment-specific configuration and .env files while keeping secrets outside the repository.

12.2 Future Improvements

CI/CD Pipeline

Implement automated:

Git Push



Build



Unit Tests



Docker Build



Security Scan



Deployment

Automated Testing

Introduce:

- Unit tests
- Integration tests
- API tests
- End-to-end tests
- Container health tests

Container Orchestration

For large-scale production deployment, containers can be managed using an orchestration platform such as Kubernetes.

This can provide:

- Automatic scaling
- Service discovery
- Self-healing
- Rolling deployments
- Load balancing

Container Security

Future deployments should implement:

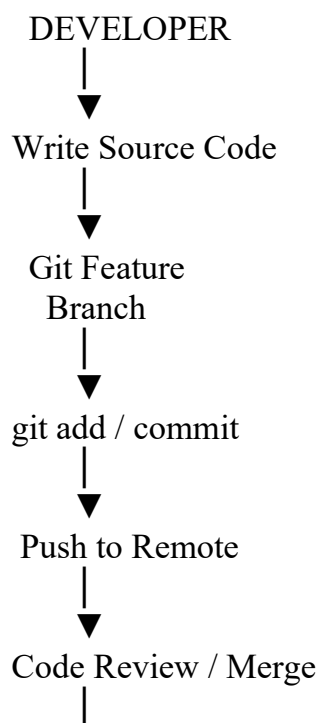
- Minimal base images
- Image vulnerability scanning
- Non-root containers
- Secret management
- Network restrictions

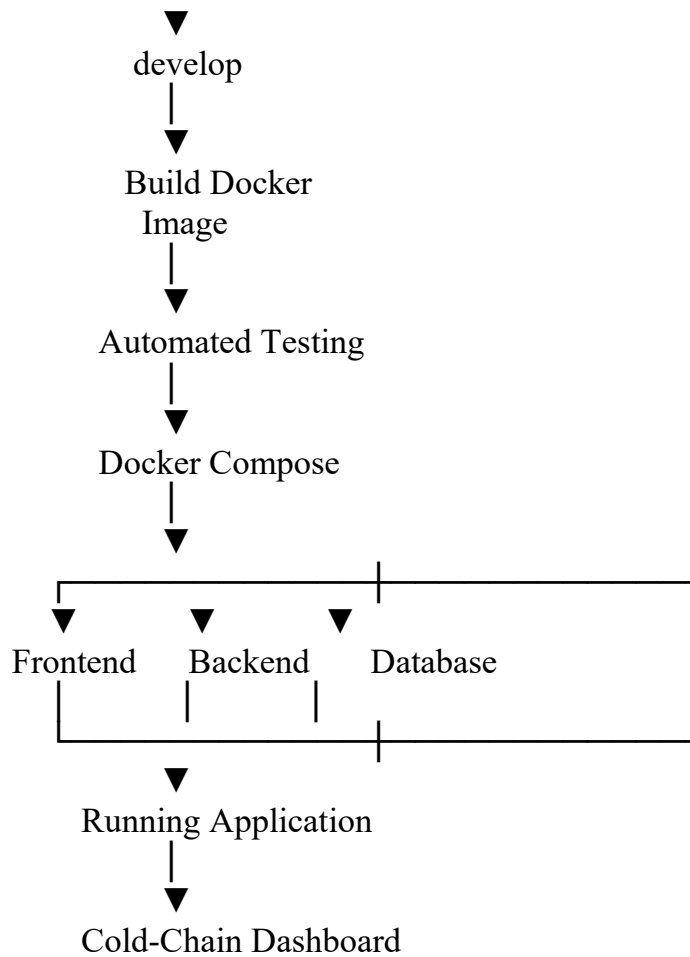
Monitoring

Implement centralized monitoring for:

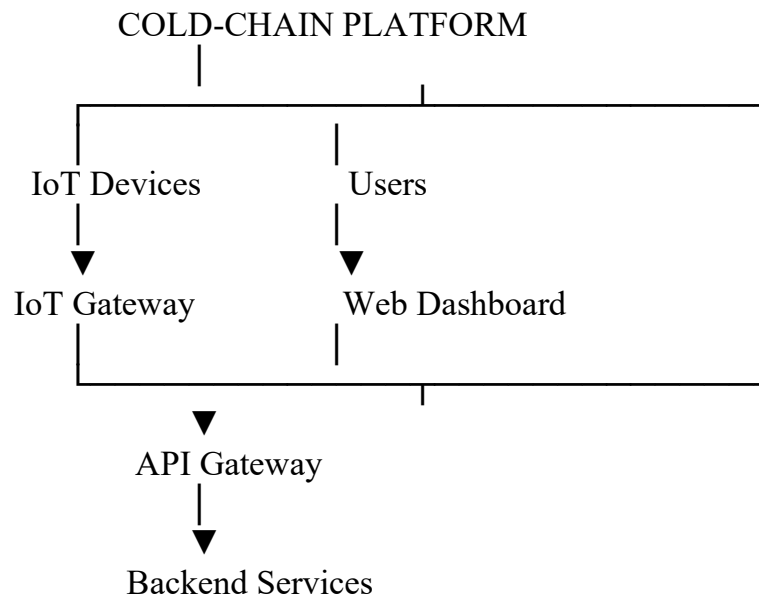
- CPU
- Memory
- Network
- Container health
- API latency
- Sensor connectivity
- Telemetry throughput

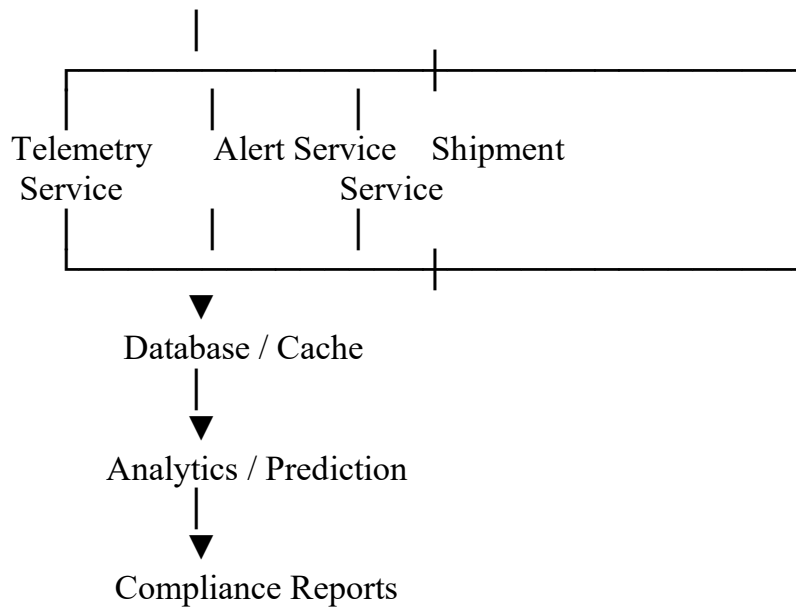
13. GIT + DOCKER WORKFLOW DIAGRAM





14. OVERALL PROJECT ARCHITECTURE





15. CONCLUSION

The **Cold-Chain Logistics Integrity Platform** requires reliable version control and a consistent deployment environment because it consists of multiple interconnected components such as IoT devices, backend services, databases, monitoring dashboards, analytics, and notification systems.

Git provides an effective version-control mechanism for managing the source code, maintaining complete project history, supporting feature-based development, resolving conflicts, and enabling collaborative development. The proposed branching strategy consisting of main, develop, feature, release, and hotfix branches provides a controlled and organized development workflow.

Docker provides containerization for the application's frontend, backend, database, and supporting services. Docker ensures that the application behaves consistently across development, testing, and deployment environments. **Docker Compose** simplifies the management of these multiple services by providing networking, environment configuration, dependencies, and persistent volumes.

The combination of Git and Docker provides a strong foundation for developing and deploying the Cold-Chain Logistics Integrity Platform. Git manages **what version of the application is being developed**, while Docker manages **how that application is executed consistently**.

Future improvements such as CI/CD automation, container orchestration, security scanning, centralized monitoring, automated testing, and predictive analytics can further improve the platform's scalability, reliability, security, and maintainability.

Therefore, the proposed Git and Docker implementation provides a **professional, collaborative, portable, scalable, and maintainable development and deployment workflow** for the Cold-Chain Logistics Integrity Platform.